

The JavaScript engine can make the computer understand JavaScript. Each browser has its own engine.

- The first JavaScript engine was created by Brendan Eich in 1995 for the Netscape Navigator Web browser, and has now evolved. It is still being used in Firefox and is called the SpiderMonkey engine.
- The first modern JavaScript engine was V8, created by Google for its Chrome browser in 2008, showing a great performance in conversion into machine code.
- Chakra is the JavaScript engine of the Internet Explorer browser. It was also forked by Microsoft for the original Edge browser, but the latter was later rebuilt as the Chromium based browser and thus now uses V8.

ECMA

By 1959, the growing use of computers, built by several different manufacturers, showed the necessity for standardisation in operation techniques, such as programming, and also input and output codes. Such standards would make it possible to use data prepared for a computer made by one manufacturer on a computer made by another with minimum changes. Also, it would avoid duplication of work in the preparation of, for example, programming languages by several manufacturers.

In a meeting held on April 27, 1960, in Brussels, it was decided that an association of manufacturers will be formed, called the European Computer Manufacturers Association or ECMA. A committee was nominated to prepare the formation of the association, and draw up bylaws and rules.

The role played by ECMA in the development of JavaScript

- In November 1996, Netscape submitted JavaScript to ECMA International, as the starting point for a standard specification that all browser vendors could conform to. This led to the official release of the first ECMAScript language specification in June 1997, which was a general-purpose, cross-platform programming language.
- The standards process continued for a few years, with the release of ECMAScript 2 in June 1998 and ECMAScript 3 in December 1999. ECMAScript 4 was released in 2000. From 2016 to 2019, a new version of the ECMAScript standard was published each year, but the scope of changes was much smaller than in the 5th and 6th editions. JavaScript is a language based on ECMAScript.
- By having standards in place, JavaScript can be executed in a similar way in all the browsers.

What is a Web server?

We get so much information from the Internet. But who provides this and how? Well, all this work is done by the Web server. These are basically simple computer programs, which handle the requests sent by different clients. The server will

respond to each client by fetching details from either one of the following:

- Another server
- Database
- File server
- Computation in the Web server – could be a CPU-intensive job

Let's take an example. We have a server that shows the current state of users affected by Covid-19 in each country. Several users or clients may request for this information at the same time. If these requests are handled serially, the users will have to wait for a very long time. Traditional Web servers like Apache and Tomcat have fork/thread mechanisms, which help to seamlessly handle many requests at the same time. Around 200+ requests can be handled in parallel from a single Web server. The requests can scale up or down based on the server configuration.

What is Node.js?

In 2009, Node.js was developed. It introduced a lot of innovative thinking for JavaScript server-side development. The Chrome V8 engine is a core component of the popular Node.js runtime system.

But didn't we say earlier that JavaScript is only for browsers? Yes, we did! Before Node.js was released, we needed a Web server and server side programming to perform actions on the Web server such as open a file, and interact with the database. To do that, we used at least one of the server programming languages such as PHP, ASP, Java, Python, etc.

But now we can have our HTTP/Web server written in JavaScript itself. It is such a saviour!! Just know JavaScript and you can become a full stack developer.

How does Node.js work?

When we have the Node.js Web server running, we also have to accept multiple clients, right? But Node.js is single-threaded, which means only one request can be handled at a time. So how do we handle multiple requests? To do so, Node.js uses two concepts.

- *Non-blocking IO*: From the client the request hits the Web server and then reaches the database. To fetch results it takes X amount of time. Rather than waiting idly, Web servers go and accept another client who has sent a request.
- *Asynchronous*: Once a client completes reading from the database, it makes a call back to the server saying it's ready, and to send its output to the client. Figure 5 shows how this is done.

From Figure 6, it can be seen how Node.js is connected to JavaScript as well as its open source project V8 and libuv.

JavaScript is the code we write. Node.js is the middleman. It executes the JS code that we've written with the help of C++, and gives us access to the APIs that are written in libuv and V8 libraries. V8 is an open source project created by